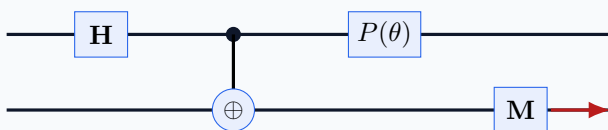


# QPU Language Reference

Complete Protocol and Truth-Table Syntax



Bailey Forbes  
September 13, 2026

The exact QPU protocol surface: references, constants, flags, gates, process composition, truth-table metadata, compatibility behavior, diagnostics, and quick-reference grammar.

# Contents

|          |                                                    |          |
|----------|----------------------------------------------------|----------|
| <b>1</b> | <b>Scope and execution model</b>                   | <b>1</b> |
| 1.1      | Three levels of support . . . . .                  | 1        |
| 1.2      | QPU instruction inventory . . . . .                | 1        |
| <b>2</b> | <b>Lexical and file syntax</b>                     | <b>1</b> |
| 2.1      | Logical lines, whitespace, and case . . . . .      | 1        |
| 2.2      | Hash comments . . . . .                            | 2        |
| 2.3      | Block comments . . . . .                           | 2        |
| 2.4      | PARAMS:: the process input declaration . . . . .   | 2        |
| 2.5      | Protocol file forms . . . . .                      | 2        |
| <b>3</b> | <b>Names and token references</b>                  | <b>3</b> |
| 3.1      | Token names . . . . .                              | 3        |
| 3.2      | Direct references . . . . .                        | 3        |
| 3.3      | Parameter references with a dollar sign . . . . .  | 3        |
| 3.4      | Cycle-qualified references with : . . . . .        | 4        |
| 3.5      | Aliases created by SET . . . . .                   | 4        |
| 3.6      | Numeric workspace references . . . . .             | 4        |
| 3.7      | Process scope and child-local references . . . . . | 5        |
| <b>4</b> | <b>Built-in state constants</b>                    | <b>5</b> |
| 4.1      | 0p: the zero basis state . . . . .                 | 5        |
| 4.2      | 1p: the one basis state . . . . .                  | 5        |
| 4.3      | sp: equal superposition . . . . .                  | 5        |
| 4.4      | Dimension-suffixed constants . . . . .             | 6        |
| 4.5      | Constants and state parameters . . . . .           | 6        |
| <b>5</b> | <b>Instruction flags and argument boundaries</b>   | <b>6</b> |
| 5.1      | -I: input and control references . . . . .         | 6        |
| 5.2      | -O: output and target references . . . . .         | 6        |
| 5.3      | Where a flag list ends . . . . .                   | 7        |
| 5.4      | No-parameter-substitution marker . . . . .         | 7        |
| 5.5      | Positional arguments . . . . .                     | 7        |
| <b>6</b> | <b>Gate instruction grammar and arity</b>          | <b>7</b> |
| 6.1      | General gate shape . . . . .                       | 7        |
| 6.2      | Why arity matters . . . . .                        | 8        |
| 6.3      | Validation and common mistakes . . . . .           | 8        |
| <b>7</b> | <b>Single-qubit primitive gates</b>                | <b>8</b> |
| 7.1      | Pauli X (X) . . . . .                              | 8        |
| 7.2      | Pauli Y (Y) . . . . .                              | 9        |
| 7.3      | Pauli Z (Z) . . . . .                              | 9        |

|           |                                                                        |           |
|-----------|------------------------------------------------------------------------|-----------|
| 7.4       | Hadamard (H) . . . . .                                                 | 9         |
| 7.5       | S phase (S) . . . . .                                                  | 10        |
| 7.6       | T phase (T) . . . . .                                                  | 10        |
| 7.7       | Arbitrary phase (PHASE) . . . . .                                      | 10        |
| 7.8       | Backward-prefix syntax . . . . .                                       | 11        |
| <b>8</b>  | <b>Controlled and two-qubit primitive gates</b>                        | <b>11</b> |
| 8.1       | Controlled NOT (CNOT) . . . . .                                        | 11        |
| 8.2       | Doubly controlled NOT / Toffoli (CCNOT) . . . . .                      | 12        |
| 8.3       | Controlled Z (CZ) . . . . .                                            | 12        |
| 8.4       | Controlled Y (CY) . . . . .                                            | 13        |
| 8.5       | Swap (SWAP) . . . . .                                                  | 13        |
| <b>9</b>  | <b>Derived logical gates</b>                                           | <b>14</b> |
| 9.1       | Logical NOT (NOT) . . . . .                                            | 14        |
| 9.2       | Reversible AND (AND) . . . . .                                         | 14        |
| 9.3       | Reversible NAND (NAND) . . . . .                                       | 15        |
| 9.4       | Reversible OR (OR) . . . . .                                           | 15        |
| 9.5       | Reversible XOR (XOR) . . . . .                                         | 15        |
| <b>10</b> | <b>Measurement gate</b>                                                | <b>16</b> |
| 10.1      | Measurement (MEASURE) . . . . .                                        | 16        |
| <b>11</b> | <b>State, token, and cycle commands</b>                                | <b>16</b> |
| 11.1      | SET — initialize or alias (lowered/executed) . . . . .                 | 16        |
| 11.2      | CREATETOKEN — reserve wires (lowered/executed) . . . . .               | 17        |
| 11.3      | DELETETOKEN — token lifetime marker (accepted, not executed) . . . . . | 17        |
| 11.4      | FREE — releasable-memory marker (accepted, not executed) . . . . .     | 17        |
| 11.5      | INCREASECYCLE — cycle boundary (lowered/executed) . . . . .            | 17        |
| 11.6      | JOIN — composite-register marker (accepted, not executed) . . . . .    | 17        |
| 11.7      | SPLIT — component-register marker (accepted, not executed) . . . . .   | 18        |
| <b>12</b> | <b>Reset: an operation that is not a gate</b>                          | <b>18</b> |
| 12.1      | Reversible gates versus RESET . . . . .                                | 18        |
| 12.2      | Where RESET comes from . . . . .                                       | 18        |
| 12.3      | Reset buttons compared . . . . .                                       | 19        |
| <b>13</b> | <b>Process composition and return values</b>                           | <b>19</b> |
| 13.1      | MAIN-PROCESS (lowered/executed) . . . . .                              | 20        |
| 13.2      | DECLARECHILD (lowered/executed) . . . . .                              | 20        |
| 13.3      | RUNCHILD — expand a child process (lowered/executed) . . . . .         | 20        |
| 13.4      | CALL — alternate child invocation (lowered/executed) . . . . .         | 20        |
| 13.5      | RETURNVALS (lowered/executed) . . . . .                                | 20        |
| 13.6      | ACCEPTVALS (lowered/executed) . . . . .                                | 21        |
| 13.7      | Nested full-adder example . . . . .                                    | 21        |

|                                                                           |           |
|---------------------------------------------------------------------------|-----------|
| <b>14 Custom gates: packaging a circuit as one block</b>                  | <b>22</b> |
| 14.1 Register one in four steps . . . . .                                 | 22        |
| 14.2 How the process maps onto wires . . . . .                            | 22        |
| 14.3 Reasoning about reversibility . . . . .                              | 22        |
| 14.4 Rules to remember . . . . .                                          | 23        |
| <b>15 Accepted compatibility commands</b>                                 | <b>23</b> |
| 15.1 COMPILEPROCESS (accepted, not executed) . . . . .                    | 23        |
| 15.2 MASTerval (accepted, not executed) . . . . .                         | 23        |
| 15.3 Save-state checkpoint instruction (accepted, not executed) . . . . . | 23        |
| 15.4 Load-state checkpoint instruction (accepted, not executed) . . . . . | 24        |
| <b>16 Truth-table companion syntax</b>                                    | <b>24</b> |
| 16.1 Text .qpuio structure . . . . .                                      | 24        |
| 16.2 MAIN-PROCESS: table header . . . . .                                 | 24        |
| 16.3 INPUTS: and OUTPUTS: column groups . . . . .                         | 24        |
| 16.4 Column header row . . . . .                                          | 24        |
| 16.5 Indexed data rows . . . . .                                          | 25        |
| 16.6 CSV spelling . . . . .                                               | 25        |
| 16.7 Partial truth tables . . . . .                                       | 25        |
| 16.8 JSON .qpuio envelope . . . . .                                       | 25        |
| 16.9 How a table test runs . . . . .                                      | 26        |
| 16.10 Protected bundled tables . . . . .                                  | 26        |
| <b>17 Compiler output and lowering details</b>                            | <b>26</b> |
| <b>18 Error conditions and practical rules</b>                            | <b>26</b> |
| <b>19 Quick-reference grammar</b>                                         | <b>27</b> |
| <b>20 Glossary</b>                                                        | <b>27</b> |
| <b>21 Operation checklist</b>                                             | <b>29</b> |

# 1 Scope and execution model

The compiler turns a plain-text `.qpucir` protocol into one flat sequence of circuit gates:

protocol source  $\longrightarrow$  logical lines  $\longrightarrow$  QPU instructions  $\longrightarrow$  scoped tokens  $\longrightarrow$  gates  $\longrightarrow$  state-vector simulation.

Child processes are expanded at compile time. The UI and simulator therefore see the same ordered gate list even when the source contains nested processes.

## 1.1 Three levels of support

| Status                        | Meaning                                                                                                                                             |
|-------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>lowered/executed</b>       | The QPU language accepts the instruction and the compiler emits behavior that the simulator executes.                                               |
| <b>accepted, not executed</b> | The QPU language accepts the instruction and preserves it in compiler output or the compiler log, but no visual/simulator operation is emitted yet. |
| <b>compiler-internal</b>      | The compiler may emit the operation itself, but protocol source cannot name it directly. RESET is the important example.                            |

## 1.2 QPU instruction inventory

| Command                     | Status                 | Purpose                                       |
|-----------------------------|------------------------|-----------------------------------------------|
| X, Y, Z, H, S, T, PHASE     | lowered/executed       | Single-qubit matrix operations.               |
| CNOT, CCNOT, CZ, CY, SWAP   | lowered/executed       | Controlled and two-wire primitive operations. |
| NOT, AND, NAND, OR, XOR     | lowered/executed       | Derived logical target mutations.             |
| MEASURE                     | lowered/executed       | Measure one named wire or all known wires.    |
| SET, CREATETOKEN            | lowered/executed       | Initialize/alias tokens and reserve wires.    |
| RUNCHILD, CALL              | lowered/executed       | Expand a catalog/library child process.       |
| RETURNVALS, ACCEPTVALS      | lowered/executed       | Bind child and process outputs.               |
| MAIN-PROCESS, INCREASECYCLE | lowered/executed       | Mark a process and flush cycle workspace.     |
| DECLARECHILD                | lowered/executed       | Bind a catalog child for later RUNCHILD/CALL. |
| FREE, DELETETOKEN           | accepted, not executed | Lifetime hints; currently logged.             |
| JOIN, SPLIT                 | accepted, not executed | Register-memory compatibility syntax.         |
| COMPILEPROCESS, MASTerval   | accepted, not executed | Process-control compatibility syntax.         |
| SAVE_STATE, LOAD_STATE      | accepted, not executed | Checkpoint compatibility syntax.              |

**Not source syntax:** RESET exists in the gate registry, but it is not a supported source instruction. Write `SET wire 0p`; the compiler schedules internal RESET gates for zeroed workspace.

# 2 Lexical and file syntax

## 2.1 Logical lines, whitespace, and case

Commands and flags are case-insensitive and are separated by whitespace. Token names retain their spelling. CRLF and LF line endings are accepted. A physical line ending *exactly* in a backslash joins to the next line:

```
PARAMS: A:state B:state \
        Cin:state
CCNOT -I A B \
      -O Carry
```

The continuation backslash must be the final character; spaces after it prevent continuation. Blank logical lines are discarded.

## 2.2 Hash comments

```
# Whole-line or trailing hash comment
H -I Q:0 -O Q:0 # prepare |+>
```

A hash starts a comment that continues through the logical line. Comments are for explanations and do not become instructions. Because continuation joining happens first, a hash on a continued logical line can hide text that appeared on a later physical line. Prefer comments after a complete instruction.

## 2.3 Block comments

```
/* A block comment may span logical lines. */
PHASE=pi/2 -I Q:0 -O Q:0
```

Text from `/*` through `*/` is ignored. A block may occupy one line or continue across several lines. Block comments are useful for temporarily annotating a region, but they should not be nested. Keep delimiters visually obvious so an omitted closing marker does not hide the remainder of a file.

## 2.4 PARAMS:: the process input declaration

```
PARAMS: A:state B:state Theta:float Count:int
```

`PARAMS:` is recognized only when it is the first non-comment logical line. Each whitespace-separated entry containing a colon becomes `name:type`. Declaration order is interface order: `RUNCHILD` inputs and truth-table input columns map from left to right.

Long declarations can use continuation:

```
PARAMS: A0:state A1:state B0:state B1:state \
        Cin:state Width:int
```

The system retains the type text:

- `state` entries become user-facing circuit inputs and truth-table input columns. Each participating state parameter resolves to a qubit wire.
- `int` and `float` entries are retained but excluded from user-facing qubit controls and truth-table state inputs. They describe numeric process metadata rather than prepared qubits.
- other type names are retained as written. Because they are not one of the recognized numeric exclusions, they may be exposed like stateful parameters; prefer `state`, `int`, or `float` for predictable behavior.

`PARAMS` has a colon after the keyword; `MAIN-PROCESS` does not. An empty or misplaced `PARAMS` line is not a substitute for a valid first-line declaration. The process's `MAIN-PROCESS` instruction is described in the composition chapter.

## 2.5 Protocol file forms

The normal authoring format is plain UTF-8 QPU text saved as `ProcessName.qpucir`. A `.qpucir.txt` download is also plain source and is useful where browsers or editors insist on a text extension. Uploading plain text derives the display name from `MAIN-PROCESS`.

The loader can also accept a JSON export envelope:

```
{
  "format": "qpucir",
  "version": 1,
  "name": "First circuit",
  "source": "MAIN-PROCESS FirstCircuit\nRETURNVALS Q"
}
```

The source string is the authoritative language text. Export envelopes may also contain compiled qubit count, gates, token map, and export timestamp metadata, but compilation from source determines current behavior.

## 3 Names and token references

A *token* is the QPU language's name for a logical storage location. For the circuits in this guide, a token normally identifies one qubit wire. Instructions refer to tokens instead of physical machine addresses. During compilation, the system resolves each live token to an integer qubit index. This indirection makes protocols readable and lets child processes have private workspace without colliding with their parents.

### 3.1 Token names

A bare token is written as a whitespace-delimited name:

```
Q
Control
CarryOut
scratch_2
0
```

Names are case-sensitive even though instruction names are not: **Carry** and **carry** are distinct tokens. A name cannot contain whitespace because whitespace separates language elements. Avoid names that begin with a hyphen: inside a flag list, any hyphen-prefixed token is treated as the beginning of another flag.

Tokens are allocated lazily. Mentioning a token in a gate, a state-setting instruction, or **CREATETOKEN** gives it a wire when compilation needs one. Consequently, source order affects the internal numeric wire index, but not the meaning of a consistently named circuit.

### 3.2 Direct references

A direct reference is simply the token's name:

```
SET Q Op
H -I Q -O Q
MEASURE -I Q
RETURNVALS Q
```

Every occurrence resolves to the same token within that process invocation. For a one-qubit gate, repeat the name after both **-I** and **-O**. This does not copy the state. It says that **Q** is the instruction's incoming wire and that the same **Q** wire is mutated in place.

### 3.3 Parameter references with a dollar sign

A leading dollar sign is the explicit parameter/reference spelling:

```
PARAMS: Input:state
MAIN-PROCESS DollarReference
SET Local $Input
H -I $Input -O $Input
RETURNVALS Local
```

Address resolution removes the leading \$. Thus `Input` and `$Input` ultimately refer to the same declared parameter. The dollar form is especially useful inside reusable child protocols because it visually separates caller-supplied values from child-local workspace.

`SET Local \ $Input` creates an alias: `Local` resolves to the same underlying wire as `Input`. It does *not* clone an unknown quantum state. Quantum states cannot generally be copied because of the no-cloning theorem; aliasing avoids claiming that a physical copy occurred.

### 3.4 Cycle-qualified references with :

A token may carry a suffix after a colon:

```
Q:0
Q:1
Carry:next
```

The current system strips the colon and everything after it when mapping source references to visual qubit wires. Therefore all three examples above whose base is `Q`—`Q:0`, `Q:1`, and bare `Q`—identify the same logical wire. The suffix is protocol notation for a stage, cycle, or bit version; it does not create a second qubit.

```
SET Q:0 Op
H -I Q:0 -O Q:1
MEASURE -I Q:2
```

This example applies `H` to one wire and then measures that same wire. The changing suffix communicates progression to a human reader, while `INCREASECYCLE` provides the actual compiler cycle boundary.

**Practical rule:** if two values must exist simultaneously, give them different base names. Do not expect `Q:0` and `Q:1` to hold two independent states.

### 3.5 Aliases created by SET

When the second operand of `SET` is not a built-in state constant, `SET` creates a name alias:

```
SET Working Input
SET Output Working
X -I Output -O Output
```

After resolution, `Working`, `Output`, and `Input` name one wire. Mutating any alias mutates that wire for every alias. This behavior is useful for readable names and process binding, but it is not assignment in the classical “copy a value into a new variable” sense.

Aliases are resolved within a process frame. `PARAMS` and parent/child bindings take precedence over ordinary child-local scoping, ensuring that an operation on a bound parameter affects the caller’s intended wire.

### 3.6 Numeric workspace references

Bare numeric names such as `0`, `1`, and `2` are valid token names and are common in reusable arithmetic children:

```
SET 0:0 $A
SET 1:0 $B
SET 2:0 $Cin
```

When a child process uses a numeric workspace name, the compiler places it in the parent invocation’s workspace namespace. This lets nested adders reuse the same compact source names without globally identifying every child invocation’s scratch wires.

Numeric token names are not number literals. `2` names a wire; it does not mean the integer two or the basis state  $|2\rangle$ .

### 3.7 Process scope and child-local references

Each process invocation receives a unique scope. A local name such as `Scratch` in two separate child calls therefore denotes two separately resolved local references unless output or parameter binding deliberately joins them. Conceptually, compilation turns:

```
RUNCHILD Worker -I A -O First
RUNCHILD Worker -I B -O Second
```

into two independent Worker frames. This is similar to local variables in two function calls. PARAMS map caller references into the frame, and RETURNVALS map selected frame references back out.

## 4 Built-in state constants

State constants describe how a wire begins. They are not ordinary token names, even though they participate in address resolution. Constants are case-insensitive and may be written with a leading dollar sign. The three supported base constants are `0p`, `1p`, and `sp`; the letter `p` distinguishes protocol state notation from ordinary numbers.

### 4.1 0p: the zero basis state

`0p` denotes

$$|0\rangle = \begin{bmatrix} 1 \\ 0 \end{bmatrix}.$$

```
SET Output 0p
```

For workspace, this schedules zero preparation. The compiler batches pending zero preparations into internal RESET operations immediately before the next real gate, at an `INCREASECYCLE`, or at process end. RESET is deliberately not source syntax; `SET Output 0p` is the public way to request zero.

Zero is the usual target initialization for reversible AND, OR, and XOR: with  $t = 0$ , the mutation  $t \leftarrow t \oplus f(x)$  leaves  $t = f(x)$ .

### 4.2 1p: the one basis state

`1p` denotes

$$|1\rangle = \begin{bmatrix} 0 \\ 1 \end{bmatrix}.$$

```
SET Flag 1p
```

The compiler realizes one preparation by applying X to a zero-initialized wire. A target beginning in one is useful when the desired reversible result is the complement of a Boolean predicate. For example,  $1 \oplus (A \wedge B) = \neg(A \wedge B)$ , which is NAND.

### 4.3 sp: equal superposition

`sp` denotes the plus state

$$|+\rangle = \frac{|0\rangle + |1\rangle}{\sqrt{2}} = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ 1 \end{bmatrix}.$$

```
SET Probe sp
```

The compiler realizes this preparation with a Hadamard gate. If immediately measured, the result is 0 or 1 with equal probability. It is not merely a hidden classical random bit: both amplitudes coexist and can interfere after later phase and Hadamard operations.

In an expected truth-table *output* cell, **sp** has a different testing role: it means either measured bit is acceptable. Context therefore matters—protocol SET initializes a plus state, while expected output metadata uses **sp** as a don’t-care marker.

## 4.4 Dimension-suffixed constants

The language also recognizes:

```
Op_dim2
1p_dim8
sp_dim4
```

The suffix must be `_dim` followed by one or more decimal digits. These forms preserve compatibility with protocols that annotate a state constant’s intended dimension. In the current simulator they still perform the same single-wire preparation as their unsuffixed forms; the dimension number does not allocate a multi-qubit register or change vector size.

## 4.5 Constants and state parameters

For an ordinary local token, SET emits preparation behavior. For a PARAMS entry typed **state**, SET supplies a default instead:

```
PARAMS: Input:state
MAIN-PROCESS DefaultInput
SET Input Op
X -I Input -O Input
RETURNVALS Input
```

The UI or truth-table runner may replace the default start state for Input. This distinction is what allows one compiled protocol to be tested over every binary input row instead of hard-wiring its parameter to zero.

# 5 Instruction flags and argument boundaries

## 5.1 -I: input and control references

**-I** begins the instruction’s input list:

```
H      -I Target      -O Target
CNOT -I Control      -O Target
CCNOT -I A B          -O Target
```

Its interpretation depends on the gate. For a one-qubit gate, the input names the same wire that the output mutates. For CNOT/CZ/CY it names one control. For CCNOT and the two-input derived logic gates it names two controls. Input order does not matter to symmetric predicates such as AND or XOR, but it remains part of the written instruction and should be kept consistent for readability.

## 5.2 -O: output and target references

**-O** begins the output list. For most gates its first token is the target wire whose amplitudes are changed:

```
X -I Q -O Q
AND -I A B -O Result
```

**-O** does not mean “allocate a fresh classical output.” The target may already contain a quantum state, and reversible logic XORs its result into that existing state. SWAP is exceptional: it requires two outputs matching its two inputs. MEASURE takes no output flag because the measurement map stores its classical result.

### 5.3 Where a flag list ends

An `-I` or `-O` list continues until the next token beginning with a hyphen or the end of the line:

```
CCNOT -I A B -O Carry -$R
```

Here A and B belong to `-I`, Carry belongs to `-O`, and `-$R` starts a separate flag. This rule is why token names should not begin with a hyphen.

Flags are case-insensitive, so `-i` and `-o` work, but uppercase is the documented style. Supplying flags in an unusual order may still parse, but `-I` followed by `-O` is the clearest and matches generated source.

### 5.4 No-parameter-substitution marker

```
PHASE=pi/4 -I Q -O Q -$R
```

The language recognizes this marker and preserves it on the instruction record as a request not to substitute parameters. The current compiler does not yet change address resolution or gate execution because of the marker. It is therefore compatibility syntax, not a reliable way to freeze a reference. Write explicit token and parameter bindings instead.

### 5.5 Positional arguments

Not every instruction uses flags. SET, MAIN-PROCESS, RETURNVALS, ACCEPTVALS, and several compatibility instructions consume positional words:

```
SET Target Op
MAIN-PROCESS Example
RETURNVALS Carry Sum
```

The command name is the first word and positional arguments follow in order. RETURNVALS order is part of the process interface. For non-gate compatibility instructions, extra words may be retained even when no execution behavior is defined; accepted text should not be mistaken for an implemented effect.

## 6 Gate instruction grammar and arity

### 6.1 General gate shape

```
GATE -I input_or_controls -O target
```

The uppercase word selects a gate. `-I` lists the incoming wire for a one-qubit operation or the controls for a controlled operation. `-O` names the wire that is mutated. Except for SWAP, gate instructions do not create a separate output wire automatically.

```
H -I Q -O Q
CNOT -I Control -O Target
```

In the first row, Q is listed twice because the state flows through H on one wire. In the second, Control is preserved and Target is conditionally changed.

## 6.2 Why arity matters

| Gate family                  | Required -I count | Required -O count |
|------------------------------|-------------------|-------------------|
| X, Y, Z, H, S, T, PHASE, NOT | 1                 | 1                 |
| CNOT, CZ, CY                 | 1                 | 1                 |
| CCNOT, AND, NAND, OR, XOR    | 2                 | 1                 |
| SWAP                         | 2                 | 2                 |
| MEASURE                      | 0 or 1            | 0                 |

All counts are exact except **MEASURE**. For ordinary single-qubit gates, name the same wire in **-I** and **-O**. The output is the mutated target; for controlled gates, **-I** contains controls.

## 6.3 Validation and common mistakes

Arity is part of the language contract. A CNOT with two inputs is not silently converted into CCNOT, and a CCNOT with one input is not converted into CNOT. The compiler reports whether an instruction has too few or too many inputs or outputs.

```
# Wrong: H needs one output.
H -I Q

# Wrong: CNOT needs exactly one control.
CNOT -I A B -O Target

# Right:
CCNOT -I A B -O Target
```

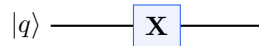
The target is removed from the compiler's control list if the same resolved wire appears on both sides. This is necessary for the normal one-qubit form. For controlled gates, do not also list the target as an input; doing so either violates arity or obscures the intended control relationship.

## 7 Single-qubit primitive gates

### Placing a single-qubit gate

X, Y, Z, H, S, T, and PHASE act on one wire, so the workbench uses only **Target particle**; **Control A** and **Control B** are ignored. PHASE also shows a **Phase angle** slider. On the canvas, drop the block on the wire you want to change.

### 7.1 Pauli X (x)



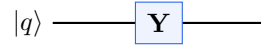
$$X = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}, \quad X|0\rangle = |1\rangle, \quad X|1\rangle = |0\rangle, \quad X(\alpha|0\rangle + \beta|1\rangle) = \beta|0\rangle + \alpha|1\rangle.$$

X is the quantum bit flip. On the Bloch sphere it is a half-turn around the x-axis. It exchanges the north and south poles and is self-inverse:  $X^2 = I$ . Unlike a classical NOT implementation, X remains meaningful on a superposition because it exchanges amplitudes linearly without measuring.

```
X -I Q:0 -O Q:0
```

| Input | Classical comparison | Output |
|-------|----------------------|--------|
| 0     | NOT 0                | 1      |
| 1     | NOT 1                | 0      |

## 7.2 Pauli Y (Y)



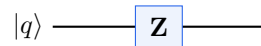
$$Y = \begin{bmatrix} 0 & -i \\ i & 0 \end{bmatrix}, \quad Y|0\rangle = i|1\rangle, \quad Y|1\rangle = -i|0\rangle.$$

It flips the basis value like X but adds phases that become observable through later interference. Geometrically it is a half-turn around the y-axis and is self-inverse,  $Y^2 = I$ . The factors  $i$  and  $-i$  are global phases for either basis state by itself, but on components of a larger superposition they may become relative phases.

Y -I Q -0 Q

| Input | Measured bit | Quantum output |
|-------|--------------|----------------|
| 0     | 1            | $i 1\rangle$   |
| 1     | 0            | $-i 0\rangle$  |

## 7.3 Pauli Z (Z)



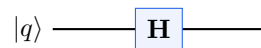
$$Z = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}, \quad Z|0\rangle = |0\rangle, \quad Z|1\rangle = -|1\rangle.$$

Z does not change a computational-basis measurement, but it changes relative phase. For example,  $Z|+\rangle = |-\rangle = (|0\rangle - |1\rangle)/\sqrt{2}$ . On the Bloch sphere it is a half-turn around z. It is self-inverse and is the simplest example showing why quantum state cannot be reduced to a table of immediate measurement probabilities.

Z -I Q -0 Q

| Input | Classical bit | Quantum output |
|-------|---------------|----------------|
| 0     | 0             | $ 0\rangle$    |
| 1     | 1             | $- 1\rangle$   |

## 7.4 Hadamard (H)



$$H = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}, \quad H|0\rangle = |+\rangle, \quad H|1\rangle = |-\rangle.$$

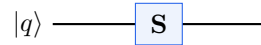
Hadamard changes between the computational z basis and the plus/minus x basis. It creates equal magnitudes from a basis state, but the sign of the second amplitude depends on the input. It is also self-inverse: applying H twice returns the original state. H is commonly used before controlled gates to create entanglement and after phase gates to reveal interference.

```
SET Q 0p
H -I Q -0 Q
MEASURE -I Q
```

| Input | Measurement distribution | State before measurement           |
|-------|--------------------------|------------------------------------|
| 0     | 50% 0, 50% 1             | $( 0\rangle +  1\rangle)/\sqrt{2}$ |
| 1     | 50% 0, 50% 1             | $( 0\rangle -  1\rangle)/\sqrt{2}$ |

Equal classical probabilities hide the relative minus sign responsible for interference.

## 7.5 S phase (S)



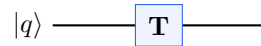
$$S = P(\pi/2) = \begin{bmatrix} 1 & 0 \\ 0 & i \end{bmatrix}, \quad S|1\rangle = i|1\rangle, \quad S^2 = Z.$$

S is a quarter-turn around the Bloch sphere's z-axis. It changes  $|+\rangle$  into  $(|0\rangle + i|1\rangle)/\sqrt{2}$  while leaving basis measurement probabilities unchanged. Its inverse is  $S^\dagger = P(-\pi/2)$ , not S itself.

```
S -I Q -0 Q
```

| Input | Measured bit | Quantum output |
|-------|--------------|----------------|
| 0     | 0            | $ 0\rangle$    |
| 1     | 1            | $i 1\rangle$   |

## 7.6 T phase (T)



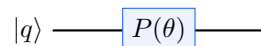
$$T = P(\pi/4) = \begin{bmatrix} 1 & 0 \\ 0 & e^{i\pi/4} \end{bmatrix}, \quad T^2 = S, \quad T^4 = Z.$$

T is an eighth-turn around z. Together with gates such as H and CNOT, it is an important non-Clifford resource for expressing general quantum computations. Its inverse is  $T^\dagger = P(-\pi/4)$ ; the current BT spelling does not synthesize that inverse, so use BPHASE=pi/4 when the negative phase is required.

```
T -I Q -0 Q
```

| Input | Measured bit | Quantum output        |
|-------|--------------|-----------------------|
| 0     | 0            | $ 0\rangle$           |
| 1     | 1            | $e^{i\pi/4} 1\rangle$ |

## 7.7 Arbitrary phase (PHASE)



$$P(\theta) = \begin{bmatrix} 1 & 0 \\ 0 & e^{i\theta} \end{bmatrix}, \quad \alpha|0\rangle + \beta|1\rangle \mapsto \alpha|0\rangle + \beta e^{i\theta}|1\rangle.$$

PHASE is the general form behind S, T, and Z. It preserves  $|\alpha|^2$  and  $|\beta|^2$  because  $|e^{i\theta}| = 1$ , yet rotates the relative phase by  $\theta$ . On the Bloch sphere this is a z-axis rotation up to global phase. A later H or another basis-changing gate is normally needed to turn that phase difference into changed computational-basis probabilities. The angle is embedded in the opcode:

```
PHASE=1.5708 -I Q -0 Q      # radians
PHASE=3/2 -I Q -0 Q        # rational radians
PHASE=90d -I Q -0 Q        # degrees
PHASE=45/2d -I Q -0 Q      # rational degrees
PHASE=pi -I Q -0 Q
PHASE=-11pi/6 -I Q -0 Q
PHASE=2*pi/3 -I Q -0 Q
BPHASE=pi/4 -I Q -0 Q      # P(-pi/4)
```

Accepted numeric atoms include signed integers, decimals, leading-dot decimals, and one optional rational denominator. Pi notation accepts an optional sign, an integer coefficient, an optional \*, and an optional positive integer denominator. A zero denominator and malformed angle raise an error.

| Input | Measured bit | Quantum output         |
|-------|--------------|------------------------|
| 0     | 0            | $ 0\rangle$            |
| 1     | 1            | $e^{i\theta} 1\rangle$ |

## 7.8 Backward-prefix syntax

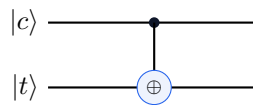
The QPU language recognizes a leading B before a supported primitive gate name, for example BX, BH, BCNOT, or BSWAP, and sets `reverse=true`. (Compiler-internal RESET remains unavailable in source.) Only BPHASE=angle currently changes emitted behavior: the compiler negates the phase angle. For every other gate the flag is metadata only. Emitting the ordinary gate still gives the mathematical inverse for these self-inverse gates:

$X, Y, Z, H, \text{CNOT}, \text{CCNOT}, \text{CZ}, \text{CY}, \text{SWAP}.$

In contrast, BS and BT do not synthesize  $S^\dagger$  or  $T^\dagger$ . Do not rely on B-prefixed forms for inverse execution unless the gate is self-inverse or is BPHASE.

# 8 Controlled and two-qubit primitive gates

## 8.1 Controlled NOT (CNOT)



$$|c, t\rangle \mapsto |c, t \oplus c\rangle.$$

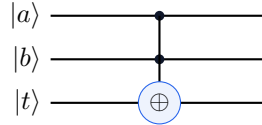
CNOT leaves the control unchanged. When the control is zero, the target is unchanged; when the control is one, X acts on the target. Because the map is a permutation of basis states, it is reversible and self-inverse. A superposition control can entangle the wires, so “classical XOR” describes only its basis-state action.

```
CNOT -I Control -0 Target
```

| $c$ | $t$ | $c'$ | $t'$ |
|-----|-----|------|------|
| 0   | 0   | 0    | 0    |
| 0   | 1   | 0    | 1    |
| 1   | 0   | 1    | 1    |
| 1   | 1   | 1    | 0    |

CNOT is a reversible embedding of XOR because it preserves the control. In the workbench, the control comes from **Control A** and the target from **Target particle**; **Control B** is ignored.

## 8.2 Doubly controlled NOT / Toffoli (CCNOT)



$$|a, b, t\rangle \mapsto |a, b, t \oplus (a \wedge b)\rangle.$$

CCNOT, also called Toffoli, applies X only when both controls are one. It is a reversible universal building block for classical computation embedded in a quantum circuit. The target's previous value matters: zero computes AND, while one computes NAND. The gate is self-inverse.

**CCNOT** -I A B -0 Target

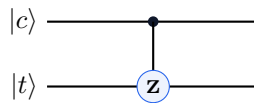
| $a$ | $b$ | $t$ | $a'$ | $b'$ | $t'$ |
|-----|-----|-----|------|------|------|
| 0   | 0   | 0   | 0    | 0    | 0    |
| 0   | 0   | 1   | 0    | 0    | 1    |
| 0   | 1   | 0   | 0    | 1    | 0    |
| 0   | 1   | 1   | 0    | 1    | 1    |
| 1   | 0   | 0   | 1    | 0    | 0    |
| 1   | 0   | 1   | 1    | 0    | 1    |
| 1   | 1   | 0   | 1    | 1    | 1    |
| 1   | 1   | 1   | 1    | 1    | 0    |

With a target initialized to zero, Toffoli computes classical AND reversibly.

### Controls are preserved, the target is the output

In the table, the  $a'$  and  $b'$  columns always equal  $a$  and  $b$ : the two controls are only read. Only  $t$  can change, and it flips exactly in the two rows where  $a = b = 1$ . In the **Interactive workbench**, choose **CCNOT** in **Gate**, the two inputs in **Control A** and **Control B**, and the output wire in **Target particle**. The *CCNOT demo* starter circuit is exactly this gate with both controls prepared as 1.

## 8.3 Controlled Z (CZ)



$$\text{CZ}|c, t\rangle = (-1)^{ct}|c, t\rangle, \quad \text{CZ} = \text{diag}(1, 1, 1, -1).$$

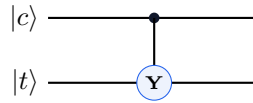
CZ adds a minus sign only to the  $|11\rangle$  amplitude. No computational-basis bit changes, but superpositions can acquire observable relative phase. CZ is symmetric between its two wires, even though QPU syntax writes one as the input control and one as the output target. It is self-inverse.

**CZ** -I Control -0 Target

| Input | Measured output | Amplitude multiplier | State         |
|-------|-----------------|----------------------|---------------|
| 00    | 00              | +1                   | $ 00\rangle$  |
| 01    | 01              | +1                   | $ 01\rangle$  |
| 10    | 10              | +1                   | $ 10\rangle$  |
| 11    | 11              | -1                   | $- 11\rangle$ |

The classical columns are unchanged; the quantum phase is the operation.

## 8.4 Controlled Y (CY)



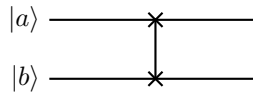
$$|0, t\rangle \mapsto |0, t\rangle, \quad |1, 0\rangle \mapsto i|1, 1\rangle, \quad |1, 1\rangle \mapsto -i|1, 0\rangle.$$

CY applies the Pauli Y rotation to the target only when the control is one. Its basis-bit pattern resembles CNOT, but the  $\pm i$  factors distinguish their behavior on superpositions. Since  $Y^2 = I$ , CY is self-inverse.

**CY** -I Control -0 Target

| $c$ | $t$ | $c'$ | $t'$ | Quantum phase |
|-----|-----|------|------|---------------|
| 0   | 0   | 0    | 0    | +1            |
| 0   | 1   | 0    | 1    | +1            |
| 1   | 0   | 1    | 1    | $i$           |
| 1   | 1   | 1    | 0    | $-i$          |

## 8.5 Swap (SWAP)



$$\text{SWAP}|a, b\rangle = |b, a\rangle.$$

SWAP exchanges the complete quantum states of two wires, including amplitudes and any correlations with the rest of the register. It does not copy either state. Three CNOTs can decompose SWAP, but the QPU system provides it as one primitive instruction. Applying SWAP twice restores the original ordering.

**SWAP** -I A B -0 A B

The two output tokens are mandatory and must resolve to the same two wires, in the same order, as the inputs.

| $a$ | $b$ | $a'$ | $b'$ |
|-----|-----|------|------|
| 0   | 0   | 0    | 0    |
| 0   | 1   | 1    | 0    |
| 1   | 0   | 0    | 1    |
| 1   | 1   | 1    | 1    |

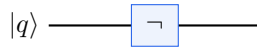
## 9 Derived logical gates

Derived gates are QPU instructions implemented as reversible target mutations. Inputs are preserved. If target  $t$  starts in zero, AND/OR/XOR write the familiar classical function. If it does not, the true behavior is the XOR embedding shown below.

### One shape for every two-input gate

AND, NAND, OR, and XOR all follow  $t' = t \oplus f(a, b)$ . The two inputs  $a$  and  $b$  (**Control A** and **Control B** in the workbench) are read and left unchanged. The target  $t$  (**Target particle**) is the output workspace and is the only wire that changes. Only  $f$  differs between the four gates. To get the ordinary classical answer, start the target at 0 with SET Out 0p.

### 9.1 Logical NOT (NOT)



$$t' = t \oplus 1.$$

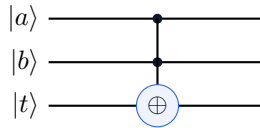
NOT is the derived logical spelling of a bit flip. In the current simulator it uses the same X matrix on the output target. The required input establishes the one-input instruction shape, but a distinct input is not copied into the output; use the same token on both sides.

NOT -I Target -0 Target

| $t$ | $t'$ |
|-----|------|
| 0   | 1    |
| 1   | 0    |

The current gate implementation mutates the output target. Use the same token for input and output; a distinct input token is not copied.

### 9.2 Reversible AND (AND)



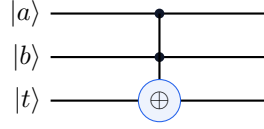
$$t' = t \oplus (a \wedge b).$$

AND flips the target exactly on the basis components where both controls are one. This is Toffoli-style behavior. It remains linear on superpositions and may entangle the target with its controls. Calling it AND is convenient only when the target is deliberately initialized.

SET Out 0p  
AND -I A B -0 Out

| $a$ | $b$ | $t = 0$ | $t' = a \wedge b$ |
|-----|-----|---------|-------------------|
| 0   | 0   | 0       | 0                 |
| 0   | 1   | 0       | 0                 |
| 1   | 0   | 0       | 0                 |
| 1   | 1   | 0       | 1                 |

### 9.3 Reversible NAND (NAND)



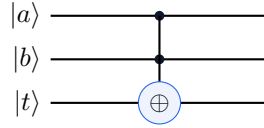
$$t' = t \oplus (a \wedge b) \oplus 1.$$

NAND first performs the all-controls-active mutation and then inverts the target. With target zero it produces the familiar NAND column. With target one it instead produces AND, illustrating why reversible target state must always be included in the full mathematical definition.

```
SET Out Op
NAND -I A B -0 Out
```

| $a$ | $b$ | $t = 0$ | $t' = \neg(a \wedge b)$ |
|-----|-----|---------|-------------------------|
| 0   | 0   | 0       | 1                       |
| 0   | 1   | 0       | 1                       |
| 1   | 0   | 0       | 1                       |
| 1   | 1   | 0       | 0                       |

### 9.4 Reversible OR (OR)



$$t' = t \oplus (a \vee b).$$

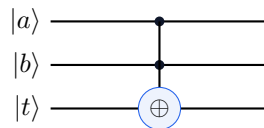
OR uses an “any control is one” predicate. If either or both controls are active, the target is flipped. This direct derived operation is equivalent on basis states to combining XOR and AND through  $a \vee b = a \oplus b \oplus (a \wedge b)$ .

```
SET Out Op
OR -I A B -0 Out
```

| $a$ | $b$ | $t = 0$ | $t' = a \vee b$ |
|-----|-----|---------|-----------------|
| 0   | 0   | 0       | 0               |
| 0   | 1   | 0       | 1               |
| 1   | 0   | 0       | 1               |
| 1   | 1   | 0       | 1               |

For bits,  $a \vee b = a \oplus b \oplus (a \wedge b)$ .

### 9.5 Reversible XOR (XOR)



$$t' = t \oplus a \oplus b.$$

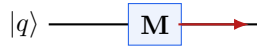
XOR flips the target when an odd number of controls is one. With two inputs, this is the ordinary exclusive-or predicate. The parity definition explains why the 11 row does not flip: two active controls have even parity.

```
SET Out Op
XOR -I A B -O Out
```

| $a$ | $b$ | $t = 0$ | $t' = a \oplus b$ |
|-----|-----|---------|-------------------|
| 0   | 0   | 0       | 0                 |
| 0   | 1   | 0       | 1                 |
| 1   | 0   | 0       | 1                 |
| 1   | 1   | 0       | 0                 |

## 10 Measurement gate

### 10.1 Measurement (MEASURE)



```
MEASURE -I Q      # exactly one named wire
MEASURE           # every wire known at this point
```

MEASURE takes no -O. Repeated one-wire commands are useful when only declared returns should become classical. A bare command emits one measurement operation per currently allocated wire.

| State                              | Outcome probabilities                 | Collapsed state            |
|------------------------------------|---------------------------------------|----------------------------|
| $ 0\rangle$                        | $P(0) = 1$                            | $ 0\rangle$                |
| $ 1\rangle$                        | $P(1) = 1$                            | $ 1\rangle$                |
| $\alpha 0\rangle + \beta 1\rangle$ | $P(0) =  \alpha ^2, P(1) =  \beta ^2$ | $ 0\rangle$ or $ 1\rangle$ |

#### Three ways to measure in the builder

**The M block** A step *in the circuit*, like any other gate. It runs every time the circuit runs and is saved in the protocol as MEASURE.

**Measure target** Measures the selected **Target particle** once, right now. Nothing is added to the circuit.

**Measure all** Measures every wire not yet measured, right now.

MEASURE, like RESET, is not reversible: once a superposition collapses, the lost amplitudes cannot be recovered. Choose **Reset state** to rewind and sample again.

## 11 State, token, and cycle commands

### 11.1 SET — initialize or alias (lowered/executed)

```
SET Q Op
SET Q 1p
SET Q sp
SET Local $Parameter
```

**SET** *target value* requires a positional value. A constant schedules or emits initialization: zero, X from zero, or H from zero. A nonconstant value makes the target an alias of that source. A **state** parameter's constant SET is treated as a runtime default, allowing the UI or truth-table runner to supply its actual start state.

**Syntax:** exactly one target and one value are meaningful. SET has no **-I** or **-O** flags. It does not perform classical assignment at each simulation step; it establishes compilation-time preparation or reference identity. See the dedicated constants and aliases chapters before using SET on unknown quantum data.

## 11.2 CREATETOKEN — reserve wires (lowered/executed)

```
CREATETOKEN -I Sum Carry Workspace
```

Every **-I** token immediately claims a stable wire index. No **-O** is used. This is useful for declared outputs before their first gate.

CREATETOKEN does not initialize amplitudes by itself. Pair new workspace with SET before relying on its value. Listing outputs early also makes process structure easier to review:

```
CREATETOKEN -I Sum Carry
SET Sum 0p
SET Carry 0p
```

## 11.3 DELETETOKEN — token lifetime marker (accepted, not executed)

```
DELETETOKEN -I Workspace
```

This says that the named token is no longer needed by the protocol. The current compiler records the instruction but does not remove, reset, or reuse its wire. It is unsafe to assume that deletion destroys sensitive state or makes a qubit available. Unused symbolic holes may disappear during final layout compaction, which is a separate implementation optimization.

## 11.4 FREE — releasable-memory marker (accepted, not executed)

```
FREE -I ScratchA ScratchB
```

FREE marks one or more input references as releasable. Like DELETETOKEN, it is currently a compatibility/lifetime hint: no physical reset, deallocation, or wire reuse occurs. Use SET with **0p** when the intended behavior is actual zero preparation for subsequent computation.

## 11.5 INCREASECYCLE — cycle boundary (lowered/executed)

```
INCREASECYCLE
```

The compiler flushes pending zero-initializations into an internal batched **RESET**, increments the cycle counter, and prepares workspace for following operations. Cycle suffixes on tokens do not create separate visual wires.

Cycles organize protocol stages. They are not wall-clock timing guarantees and do not cause gates to execute concurrently. The final circuit is still an ordered gate sequence. Their practical implemented effect today is the workspace-zero boundary.

## 11.6 JOIN — composite-register marker (accepted, not executed)

```
JOIN -I A B -O AB
```

JOIN expresses the intent to view smaller registers as one composite register. For independent inputs, the mathematical construction is

$$|AB\rangle = |A\rangle \otimes |B\rangle.$$

Tensor product increases dimension multiplicatively: two one-qubit vectors of length two produce one two-qubit vector of length four. The current compiler logs JOIN but emits no tensor-product allocation, alias, or visible gate. Because JOIN is not a gate instruction, its input/output count is not currently validated.

## 11.7 SPLIT — component-register marker (accepted, not executed)

```
SPLIT AB A 2
```

SPLIT expresses the reverse organizational intent: identify a component of a larger register. A general entangled state cannot be factored into independent pure states, so “splitting” is not equivalent to slicing an ordinary array. For example,  $(|00\rangle + |11\rangle)/\sqrt{2}$  has no separate pure state for A. The current compiler logs the positional arguments but performs no extraction, measurement, or aliasing. Code requiring an implemented result should instead operate directly on named wires and RETURNVALS.

## 12 Reset: an operation that is not a gate

The word *reset* appears in two unrelated places: the compiler’s internal RESET operation, and three buttons in the Circuit builder. Neither is an ordinary quantum gate.

### 12.1 Reversible gates versus RESET

Every gate in the palette except **M** is *reversible*: its effect can always be undone. X, Y, Z, H, CNOT, CCNOT, CZ, CY, SWAP, NOT, AND, NAND, OR, and XOR undo themselves when applied a second time. S, T, and PHASE are undone by the same phase with the opposite sign. Mathematically each is a *unitary* matrix  $U$ , and  $U^\dagger$  undoes it.

RESET is different. It forces one wire to  $|0\rangle$  whatever it held before:

$$|0\rangle \mapsto |0\rangle, \quad |1\rangle \mapsto |0\rangle, \quad \alpha|0\rangle + \beta|1\rangle \mapsto |0\rangle.$$

Two different inputs,  $|0\rangle$  and  $|1\rangle$ , give the same output, so nothing can tell afterwards which one was there. The information has been erased, and no matrix  $U$  can undo that. RESET is therefore not unitary, just like MEASURE.

|                                  | Reversible gates (X, H, MEASURE<br>CNOT, AND, ...) |                            | RESET                  |
|----------------------------------|----------------------------------------------------|----------------------------|------------------------|
| <b>Unitary?</b>                  | Yes                                                | No                         | No                     |
| <b>Can be undone?</b>            | Yes, by the inverse gate                           | No                         | No                     |
| <b>Result depends on chance?</b> | No                                                 | Yes                        | No                     |
| <b>Written in source?</b>        | Yes                                                | Yes                        | No — write SET wire 0p |
| <b>Shown on canvas?</b>          | Yes                                                | Yes, as the meter <b>M</b> | No (hidden)            |

### 12.2 Where RESET comes from

You never type RESET. The compiler inserts it when a wire must start at zero:

- SET Out 0p on a workspace wire. (SET Out 1p and SET Out sp are instead prepared by applying X or H to a wire that starts at 0.)
- INCREASECYCLE, which zeroes pending workspace before the next stage.
- RUNCHILD and CALL, which zero each parent output wire once before expanding the child.

RESET steps are hidden from the canvas and filtered from the **Execution log** so that the visible circuit shows only the gates you wrote. They still run during simulation.

### Reset a wire only when it is not entangled

In this simulator, RESET keeps the part of the state in which the wire is already 0 and rescales it; only if that part is empty does it move the  $|1\rangle$  amplitudes to  $|0\rangle$ . On a fresh wire this is exactly “prepare  $|0\rangle$ ”. On a wire entangled with others it also changes the other wires. For example, resetting one half of a Bell pair forces the other half to 0 as well. In multi-stage circuits, *uncompute* temporary wires back to 0 with reversible gates instead of relying on a reset.

## 12.3 Reset buttons compared

None of these buttons inserts a RESET operation into your circuit. They only change what the page is showing and simulating.

| Kept or cleared?                  | Reset state | Clear circuit | Reset site | Notes                                                                      |
|-----------------------------------|-------------|---------------|------------|----------------------------------------------------------------------------|
| Gates on the canvas               | kept        | cleared       | cleared    |                                                                            |
| Number of wires                   | kept        | kept          | back to 3  |                                                                            |
| Start states ( <i>name</i> start) | kept        | kept          | all 0p     |                                                                            |
| Protocol editor text              | kept        | rewritten     | default    | Clear circuit writes an empty canvas protocol into the editor.             |
| Compiled PARAMS / RETURNVALS      | kept        | cleared       | cleared    |                                                                            |
| Quantum state and measurements    | rewound     | rewound       | rewound    | Back to the start states; log restarts.                                    |
| Workbench selections              | kept        | kept          | defaults   | Gate H, target q0, controls q1 and q2, phase 90°.                          |
| Custom gates and catalog          | kept        | kept          | kept       | Nothing removes these except <b>Remove</b> or closing the browser session. |

### Which one do I want?

**Reset state** “Run the same experiment again.” Use it between repeated runs of a circuit that measures, to collect several samples.

**Clear circuit** “Keep my wires, start a new circuit.”

**Reset site** “Put everything back the way it was when I first opened the page.” The **Reset site** entry in the navigation menu and **Reset site completely** on the **More** page do the same thing.

## 13 Process composition and return values

### Main process and child process in plain words

A **process** is a named list of gates with inputs (PARAMS) and outputs (RETURNVALS). The **main process** is the one you compile and run: the name after MAIN-PROCESS. A **child process** is another saved process that the main process borrows with RUNCHILD, the way a program calls a helper function.

The difference from a function call is that nothing is called at run time. The compiler copies the child’s gates into the main circuit, on the wires you pass: **-I** wires feed the child’s inputs in PARAMS order, and **-O** wires receive its outputs in RETURNVALS order. The canvas therefore shows every gate, including the ones that came from children.

“Main” and “child” are roles in one compile, not kinds of file. **SingleBitFullAdder** is the main process when you run it on its own and a child when **TwoBitFullAdder** runs it twice. A custom gate is the canvas version of the same idea: a saved process placed on wires you choose.

### Browse a process and its children

In the Circuit builder, press the `TwoBitFullAdder` protocol button. The Interactive workbench gains a **Compiled protocol** card showing the row of its truth table that matches the current start states. Open **Learn more** on that card for the full 32-row table (grouped by the leading input bits), its notes, and a `SingleBitFullAdder` entry you can expand in place to read the child.

## 13.1 MAIN-PROCESS (lowered/executed)

```
MAIN-PROCESS FullAdder
```

This marks the process body and supplies its catalog/library name. Compilation already begins from the named process, so the row itself emits no gate. The name is the first positional word after `MAIN-PROCESS`; avoid spaces inside it. This name connects a protocol to catalog entries, child calls, downloads, and companion truth-table metadata. If no marker exists, the system uses `InlineProcess`, but named processes are strongly recommended.

## 13.2 DECLARECHILD (lowered/executed)

```
DECLARECHILD SingleBitFullAdder
```

The named child is bound from the compilation `librarySources` catalog. An unknown name fails on this line. The row itself emits no gate; later `RUNCHILD` or `CALL` expansions use the bound process body. A successful declaration therefore guarantees that the named child exists in the catalog supplied to this compile, and that later calls of that name expand the bound body rather than a later alias.

## 13.3 RUNCHILD — expand a child process (lowered/executed)

```
RUNCHILD SingleBitFullAdder \
  -I A B Cin \
  -O Carry Sum
```

The first positional argument is the child process name. Inputs bind by the child's `PARAMS` order: `A` maps to the first parameter, `B` to the second, and `Cin` to the third in this example. Outputs bind by the child's `RETURNVALS` order. Before expansion, each bound parent output is zero-prepared once. Child-local names receive a unique scope.

`RUNCHILD` is compile-time composition, not a dynamic subroutine call in a running quantum machine. The child body is recursively expanded into the parent's flat gate sequence. This allows the renderer and simulator to inspect every resulting gate.

## 13.4 CALL — alternate child invocation (lowered/executed)

```
CALL SingleBitFullAdder -I A B Cin -O Carry Sum
```

`CALL` currently follows the same child lookup, parameter binding, output preparation, and expansion path as `RUNCHILD`. `RUNCHILD` is used by the bundled multi-bit protocols and makes compile-time expansion explicit; `CALL` is retained as an equivalent system spelling. Both fail when the named child is absent from the supplied process library.

## 13.5 RETURNVALS (lowered/executed)

```
RETURNVALS Carry Sum
```

Positional tokens define the process's ordered public outputs. They determine child output binding, truth-table output columns, and the UI's preferred logical width. Do not use `-I` or `-O` here: flags would be retained as positional return tokens.

RETURNVALS does not measure automatically. Add MEASURE instructions when classical outcomes are required. Returning an unmeasured wire preserves it as part of the projected quantum output shown by the system.

### 13.6 ACCEPTVALS (lowered/executed)

```
CALL Child -I A B
ACCEPTVALS LocalCarry LocalSum
```

Each positional local token aliases the corresponding value from the most recent child return list. Direct `-0` binding on RUNCHILD/CALL is clearer when output names are known.

Ordering is again essential: the first ACCEPTVALS name receives the first returned reference, and so on. The instruction refers to the most recently completed child expansion in compiler state; inserting another child call changes which return list is accepted.

### 13.7 Nested full-adder example

```
# child.qpucir
PARAMS: A:state B:state Cin:state
MAIN-PROCESS ChildAdder
CREATETOKEN -I Carry Sum
SET Carry Op
SET Sum Op
XOR -I A B -O Sum
CNOT -I Cin -O Sum
CCNOT -I A B -O Carry
CCNOT -I A Cin -O Carry
CCNOT -I B Cin -O Carry
RETURNVALS Carry Sum

# parent.qpucir
PARAMS: A:state B:state Cin:state
MAIN-PROCESS Parent
CREATETOKEN -I Cout Result
RUNCHILD ChildAdder -I A B Cin -O Cout Result
MEASURE -I Cout
MEASURE -I Result
RETURNVALS Cout Result
```

For basis inputs, the full-adder equations are

$$\text{Sum} = A \oplus B \oplus C_{\text{in}}, \quad C_{\text{out}} = (A \wedge B) \oplus (A \wedge C_{\text{in}}) \oplus (B \wedge C_{\text{in}}).$$

| <i>A</i> | <i>B</i> | <i>C<sub>in</sub></i> | <i>C<sub>out</sub></i> | <i>Sum</i> |
|----------|----------|-----------------------|------------------------|------------|
| 0        | 0        | 0                     | 0                      | 0          |
| 0        | 0        | 1                     | 0                      | 1          |
| 0        | 1        | 0                     | 0                      | 1          |
| 0        | 1        | 1                     | 1                      | 0          |
| 1        | 0        | 0                     | 0                      | 1          |
| 1        | 0        | 1                     | 1                      | 0          |
| 1        | 1        | 0                     | 1                      | 0          |
| 1        | 1        | 1                     | 1                      | 1          |

## 14 Custom gates: packaging a circuit as one block

A custom gate is a whole QPU process saved under a short name so that it can be placed like any built-in block. It adds no new physics: when it runs, the simulator replays the process's own gates, one by one, on the wires you chose. Understanding a custom gate therefore means understanding the smaller gates inside it.

### 14.1 Register one in four steps

1. Paste the process below into the protocol editor in **Compile text into a circuit** and choose **Compile protocol** to confirm it has no errors.
2. In **Register custom gates from QPU processes**, type NOR in **Gate id** (palette label) and leave **Source process** on *Current protocol editor*.
3. Choose **Register custom gate**. A NOR block appears under **Custom gates** in the palette and in the workbench **Gate** list.
4. Place it with **Control A**, **Control B**, and **Target particle**, exactly like AND.

Complete runnable protocol

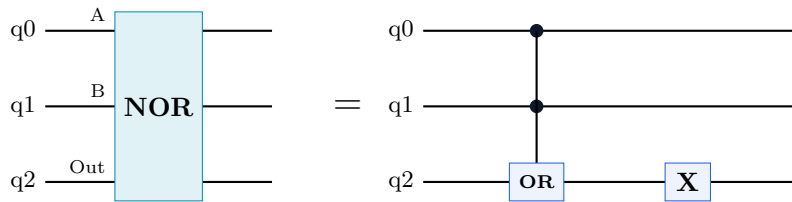
```
PARAMS: A:state B:state

MAIN-PROCESS NorGate
CREATETOKEN -I Out

OR -I A B -O Out
X -I Out -O Out

RETURNVALS Out
```

### 14.2 How the process maps onto wires



| In the process       | Becomes, when the block is placed                                   |
|----------------------|---------------------------------------------------------------------|
| 1st PARAMS entry     | <b>Control A</b> (the lowest free wire when dropped on the canvas). |
| 2nd PARAMS entry     | <b>Control B</b> . Further entries take the next free wires.        |
| 1st RETURNVALS entry | <b>Target particle</b> : the only wire the block should change.     |
| Any other token      | A fresh extra workspace wire that the block allocates for itself.   |

The palette gives every custom block exactly one target wire, so register processes with a single **RETURNVALS** output. A process with two outputs, such as a half-adder, compiles on its own but cannot be placed as a block; use it through **RUNCHILD** instead.

### 14.3 Reasoning about reversibility

A sequence of reversible gates is reversible: undo the last gate first, then the one before it, and so on.

$$(U_k \cdots U_2 U_1)^{-1} = U_1^{-1} U_2^{-1} \cdots U_k^{-1}.$$

NorGate is OR followed by X. Both undo themselves, so NOR undoes itself too: placing the block twice in a row returns every wire to where it started. Written as a rule, the block performs

$$\text{Out}' = \text{Out} \oplus \neg(A \vee B),$$

the same “target XOR function” shape as AND. That shape is always reversible, because applying it again XORs the same value back out.

### Three things that make a custom gate irreversible

**MEASURE inside the process** The block collapses its wires every time it runs.

**SET on the output** SET Out 0p makes the compiler insert a hidden RESET, so the block *overwrites* the target ( $\text{Out}' = f(A, B)$ ) instead of XORing into it. Whatever the target held before is erased. NorGate deliberately has no SET: a freshly compiled wire already starts at 0, so it passes the same truth table.

**Temporary wires left dirty** Helper tokens become hidden workspace wires. If a helper is not returned to 0 (uncomputed), it stays entangled with the inputs, and the visible wires alone no longer behave reversibly.

### A quick test you can run in the builder

Place your block twice on the same wires and choose **Run all**. For a reversible out-of-place gate, every wire ends exactly as it started. Try it with the target’s *name* start set to 0p and then 1p. If the target does not return, look for one of the three problems above.

## 14.4 Rules to remember

- The gate id must start with a letter, use only letters, digits, and underscores, and cannot match a built-in gate such as AND.
- Custom gates are saved for the current browser session only, not in the bundled catalog. Keep the .qpucir file to recreate one later.
- A custom gate can wrap a process that itself uses RUNCHILD; the child processes are captured when you register it.

## 15 Accepted compatibility commands

These instructions are part of the accepted QPU language, but their present effect is to preserve arguments and add a compiler-log entry. Their acceptance must not be confused with simulator behavior.

### 15.1 COMPILEPROCESS (accepted, not executed)

**COMPILEPROCESS** ChildName

COMPILEPROCESS records a request associated with another process. It does not currently locate, compile, or insert that process. Use RUNCHILD or CALL with a process available in the compilation library when actual child gates are required. Arguments are positional and no arity is enforced.

### 15.2 MASTERVAL (accepted, not executed)

**MASTERVAL** Result

MASTERVAL records the intent that a reference is a final or authoritative value. It does not change the process interface, measurement behavior, token mapping, or RETURNVALS. Use RETURNVALS to expose outputs that the current system will actually project and associate with truth-table columns.

### 15.3 Save-state checkpoint instruction (accepted, not executed)

**SAVE\_STATE** checkpointA

SAVE\_STATE records a named checkpoint request. The simulator does not currently copy the state vector into a checkpoint store. In particular, later gates and measurements proceed exactly as if this line were absent. A log entry is not evidence that a recoverable snapshot exists.

## 15.4 Load-state checkpoint instruction (**accepted, not executed**)

```
LOAD_STATE checkpointA
```

LOAD\_STATE records a restore request but does not replace the active state vector. It neither validates that a matching SAVE\_STATE appeared nor rewinds measurements. Protocols whose correctness depends on restoration must wait for runtime support rather than relying on accepted syntax.

## 16 Truth-table companion syntax

Truth tables are metadata paired to a process; they are not `.qpucir` commands. A protocol describes *how* a circuit works; its companion `.qpuio` describes selected input/output expectations.

### 16.1 Text `.qpuio` structure

A text table may use spaces or commas:

```
MAIN-PROCESS: HalfAdder
INPUTS: A B
OUTPUTS: Carry Sum
# A B Carry Sum
0 0p 0p 0p 0p
1 0p 1p 0p 1p
2 1p 0p 0p 1p
3 1p 1p 1p 0p
```

### 16.2 MAIN-PROCESS: table header

The colon is required in the table format:

```
MAIN-PROCESS: HalfAdder
```

This name associates metadata with its protocol. Notice the deliberate syntax difference: protocol source writes `MAIN-PROCESS HalfAdder`, while the truth-table header writes `MAIN-PROCESS: HalfAdder`.

### 16.3 INPUTS: and OUTPUTS: column groups

```
INPUTS: A B
OUTPUTS: Carry Sum
```

These lines divide the columns into values supplied before simulation and values checked after simulation. Their order must match the later column header. INPUTS should correspond to state-typed PARAMS; OUTPUTS should correspond to RETURNVALS. OUTPUTS must contain at least one name.

Explicit groups remove ambiguity. If they are absent, the loader can derive the split only when a matching protocol is available and the combined PARAMS and RETURNVALS widths match the table header.

### 16.4 Column header row

```
# A B Carry Sum
```

The first nonmetadata row beginning with a hash and whitespace or comma is the ordered column header. Here A and B are inputs, followed by Carry and Sum outputs. Unlike an ordinary protocol hash comment, this row has structural meaning inside a `.qpuio` file.

## 16.5 Indexed data rows

```
0 0p 0p 0p 0p
1 0p 1p 0p 1p
```

The first cell is a decimal row index. Indices begin at zero and increase by one without gaps. Remaining cells align left-to-right with the column header. Every row must have exactly the expected width, and every state cell must be 0p, 1p, or sp.

For  $n$  binary input columns, a complete table contains  $2^n$  rows. The usual ordering interprets input columns most-significant first: for two inputs the sequence is 00, 01, 10, 11. The loader checks sequential row indices, while semantic validation also rejects duplicate input patterns.

## 16.6 CSV spelling

Commas may replace whitespace consistently:

```
MAIN-PROCESS: HalfAdder
INPUTS: A,B
OUTPUTS: Carry,Sum
#, A, B, Carry, Sum
0,0p,0p,0p,0p
```

Spaces around commas are trimmed. Do not mix extra prose into a data row; trailing hash comments are allowed only where the table loader can strip them after the data content.

## 16.7 Partial truth tables

A partial table lists selected cases rather than all  $2^n$  combinations. It must contain at least one row, no duplicate input pattern, and no more than  $2^n$  rows. Partial tables are useful for state-machine transitions or focused regressions, but passing one proves behavior only for the listed cases.

| Input count $n$ | Complete row count $2^n$ | Example                 |
|-----------------|--------------------------|-------------------------|
| 0               | 1                        | constant-output process |
| 1               | 2                        | 0, 1                    |
| 2               | 4                        | 00, 01, 10, 11          |
| 3               | 8                        | 000 through 111         |
| 4               | 16                       | 0000 through 1111       |

## 16.8 JSON .qpuio envelope

The equivalent machine-oriented JSON form is:

```
{
  "format": "qpuio",
  "version": 1,
  "processName": "HalfAdder",
  "inputColumns": ["A", "B"],
  "outputColumns": ["Carry", "Sum"],
  "rows": [
    ["0p", "0p", "0p", "0p"],
    ["0p", "1p", "0p", "1p"],
    ["1p", "0p", "0p", "1p"],
    ["1p", "1p", "1p", "0p"]
  ]
}
```

The **format** value must be **qpuio** and **version** must be 1. Column names must be nonempty strings, outputColumns must not be empty, and every nested row array must have exactly

$$\text{inputColumns.length} + \text{outputColumns.length}$$

valid cells.

## 16.9 How a table test runs

For each listed row, the runner:

1. maps input cells to state PARAMS by column name and order;
2. creates the circuit's start-state vector;
3. executes every compiled gate, including child expansions;
4. measures the declared RETURNVALS;
5. compares only those measured outputs with expected output cells.

An expected output **sp** accepts either measured bit. An input **sp** initializes an equal plus-state superposition. Because one probabilistic sample cannot establish a full distribution, tables are strongest for deterministic basis-state behavior.

## 16.10 Protected bundled tables

The bundled SingleBitFullAdder, TwoBitFullAdder, FourBitFullAdder, and PhaseDemo tables are canonical site assets. Do not rewrite them to make an experiment pass. Create a new uploaded or corrected process/table pair when testing different expectations. This keeps examples reproducible and prevents a protocol correction from silently changing its specification.

# 17 Compiler output and lowering details

Compilation returns:

```
CompileResult {
  gates: CircuitGate[]
  qubitCount: number
  logicalQubitCount: number
  parsed: ParsedCommand[]
  log: string[]
  tokenMap: Record<string, number>
  processParams: ProcessParam[]
  returnValues: ReturnValue[]
}
```

Each accepted instruction entry records **op**, original **raw** text, **inputs**, **outputs**, positional **args**, optional normalized radian **phase**, **reverse**, and **noParameterSubstitution**. Each emitted gate records **id**, **type**, ordered **step**, **targets**, **controls**, optional **phase**, and source text.

After child expansion, unused symbolic holes are compacted to dense qubit indices. Internal RESET gates count for simulation but are filtered from the visible circuit. UI logical width prefers RETURNVALS count, then exposed PARAMS count, then physical simulator width.

# 18 Error conditions and practical rules

1. Unknown opcodes fail with **Unknown command**. RESET is not a source opcode.
2. Every QPU gate except MEASURE requires **-I** and **-O**, then exact registry arity is checked.
3. PHASE requires a valid nonzero-denominator angle after **=**.
4. SWAP requires two matching input/output wires.
5. RUNCHILD/CALL fails if the named process is absent from the library. Unknown DECLARECHILD names fail on the declaration line.

6. SET fails when its positional value is missing.
7. RETURNVALS order is semantically significant for child bindings and truth-table columns.
8. Use the same input/output token for single-qubit gates.
9. Initialize derived-gate targets deliberately; their operation is XOR into the existing target, not destructive assignment.
10. Treat accepted-but-not-executed instructions as compatibility annotations until their runtime lowering is implemented.

## 19 Quick-reference grammar

This is a practical description of the accepted surface language rather than a separate formal language specification:

```

protocol      := [params-line] logical-line*
params-line   := "PARAMS:" (name ":" type)*
logical-line  := command | comment | blank

command       := opcode argument*
opcode        := supported-command
               | primitive-gate
               | "B" primitive-gate
               | "PHASE=" angle
               | "BPHASE=" angle

gate-row      := opcode "-I" input+ "-O" output+ ["-$R"]
measure-row   := "MEASURE" ["-I" input]
set-row       := "SET" target value
process-row   := "MAIN-PROCESS" name
child-row     := ("RUNCHILD" | "CALL") name
               ["-I" argument*] ["-O" argument*]
returns-row   := "RETURNVALS" token*
accept-row    := "ACCEPTVALS" token*

token         := ["$"] name [":" suffix]
constant      := ("0p" | "1p" | "sp") ["_dim" integer]
angle         := rational
               | rational "d"
               | [sign] [integer ["*"]] "pi" ["/" integer]

```

**Syntax detail:** PHASE= pi/2 is joined into one opcode/value pair, but PHASE = pi/2 is not. Operations and flags are case-insensitive; names, types, and positional values are otherwise retained.

## 20 Glossary

| Term                   | Meaning                                                                                                                                                       |
|------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Amplitude</b>       | A complex coefficient of a basis state. Its squared magnitude contributes to measurement probability.                                                         |
| <b>Ancilla</b>         | An auxiliary qubit or workspace wire, usually prepared in a known state, used to hold intermediate information in a reversible computation.                   |
| <b>Basis state</b>     | A state used as a coordinate direction. The computational basis for one qubit is $ 0\rangle,  1\rangle$ .                                                     |
| <b>Bloch sphere</b>    | A geometric representation of pure one-qubit states after ignoring global phase. North/south are $ 0\rangle/ 1\rangle$ ; longitude represents relative phase. |
| <b>Boolean algebra</b> | The mathematics of 0/1 values and operations such as NOT, AND, OR, and XOR.                                                                                   |
| <b>Child process</b>   | A separately named protocol expanded into a parent through RUNCHILD or CALL.                                                                                  |

| Term                       | Meaning                                                                                                                                                                                            |
|----------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Combinational logic</b> | A classical circuit whose output depends only on current inputs, not stored history.                                                                                                               |
| <b>Control</b>             | A wire whose basis condition decides whether a controlled operation acts on its target. The control itself is preserved by the standard controlled gates.                                          |
| <b>Entanglement</b>        | A joint multi-qubit state that cannot be written as a tensor product of independent qubit states.                                                                                                  |
| <b>Fan-out</b>             | Sending one classical value to multiple destinations. Unknown quantum states cannot be universally copied; CNOT copies only known basis information onto a zero target and otherwise may entangle. |
| <b>Gate</b>                | A state transformation. Closed-system quantum gates are unitary; measurement and reset have different semantics.                                                                                   |
| <b>Garbage</b>             | Reversible-workspace information no longer wanted as output but still present because it cannot simply be erased.                                                                                  |
| <b>Global phase</b>        | A unit complex multiplier applied to the whole state. It does not change physical measurement predictions.                                                                                         |
| <b>Ket</b>                 | Dirac notation $ \psi\rangle$ for a state column vector.                                                                                                                                           |
| <b>Measurement</b>         | Conversion of quantum amplitudes into a sampled classical result, with state collapse and renormalization.                                                                                         |
| <b>Phase</b>               | The angular part of a complex amplitude. Relative phase controls interference even when immediate basis probabilities are unchanged.                                                               |
| <b>Process</b>             | A named QPU protocol body with ordered PARAMS and RETURNVALS.                                                                                                                                      |
| <b>Protocol</b>            | Plain-text QPU instructions describing preparation, gates, composition, measurement, and returned references.                                                                                      |
| <b>Qubit</b>               | A normalized two-component complex state $\alpha 0\rangle + \beta 1\rangle$ , not merely an unknown classical bit.                                                                                 |
| <b>Register</b>            | One or more logical wires considered together. Current JOIN/SPLIT syntax is accepted but does not yet lower to register operations.                                                                |
| <b>Relative phase</b>      | The phase difference between amplitudes. Unlike global phase, it can change later interference and measurement probabilities.                                                                      |
| <b>Superposition</b>       | A coherent linear combination of basis states. <code>sp</code> prepares the equal plus superposition.                                                                                              |
| <b>Target</b>              | The wire whose amplitudes a gate mutates; normally the first <code>-0</code> reference.                                                                                                            |
| <b>Tensor product</b>      | The operation $\otimes$ that combines state spaces. Two one-qubit vectors produce a four-amplitude two-qubit vector.                                                                               |
| <b>Token</b>               | A source-level logical name resolved to a qubit wire or alias during compilation.                                                                                                                  |
| <b>Truth table</b>         | Listed input/output expectations for measured RETURNVALS. It describes basis behavior but not arbitrary phase or entanglement.                                                                     |
| <b>Uncomputation</b>       | Running suitable inverse gates in reverse order to return temporary workspace to a known state without irreversible erasure.                                                                       |
| <b>Unitary</b>             | A matrix satisfying $U^\dagger U = I$ , preserving normalization and having a reversible inverse.                                                                                                  |

## 21 Operation checklist

| Item           | Covered in           | Item             | Covered in                  |
|----------------|----------------------|------------------|-----------------------------|
| PARAMS:        | Lexical syntax       | MAIN-PROCESS     | Process composition         |
| X,Y,Z,H        | Single-qubit gates   | S,T,PHASE        | Single-qubit gates          |
| CNOT,CCNOT     | Controlled gates     | CZ,CY,SWAP       | Controlled gates            |
| NOT,AND,NAND   | Derived gates        | OR,XOR           | Derived gates               |
| MEASURE        | Measurement          | SET              | State/token commands        |
| CREATETOKEN    | State/token commands | DELETETOKEN,FREE | State/token commands        |
| INCREASECYCLE  | State/token commands | JOIN,SPLIT       | State/token commands        |
| DECLARECHILD   | Process composition  | RUNCHILD,CALL    | Process composition         |
| RETURNVALS     | Process composition  | ACCEPTVALS       | Process composition         |
| COMPILEPROCESS | Compatibility        | MASTerval        | Compatibility               |
| SAVE_STATE     | Compatibility        | LOAD_STATE       | Compatibility               |
| -I,-O,-\$R     | Flags                | B prefix         | Single-qubit/reverse syntax |
| comments,\     | Lexical syntax       | .qpuio           | Truth-table syntax          |